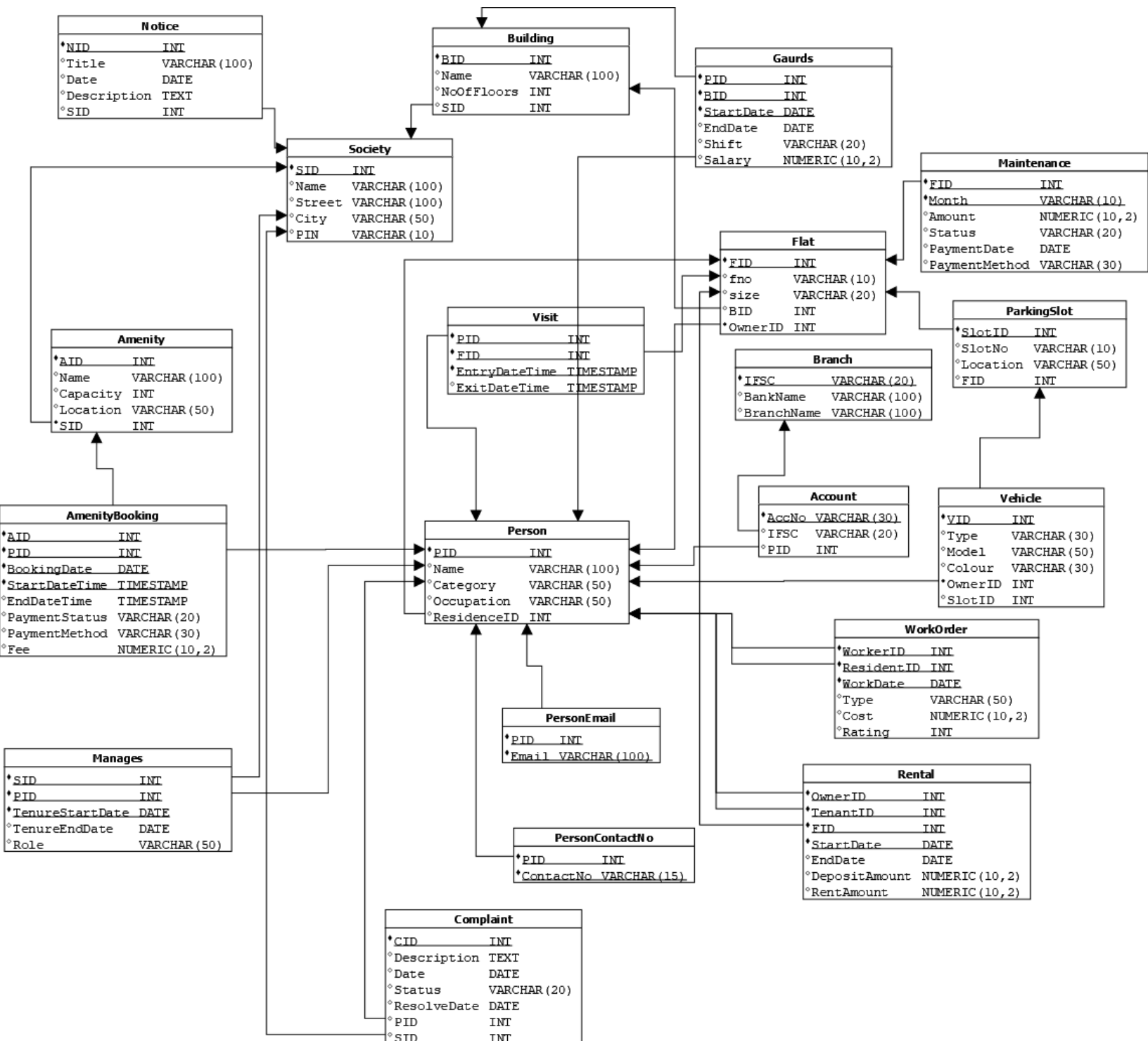


1. Relational Schema Diagram:



2. Minimal FD Set:

Society

sid $\rightarrow$ name

sid $\rightarrow$ street

sid $\rightarrow$ city

sid $\rightarrow$ pin

Building

bid $\rightarrow$ name

bid $\rightarrow$ nooffloors

bid $\rightarrow$ sid

Notice

nid $\rightarrow$ title

nid $\rightarrow$ date

nid $\rightarrow$ description

nid $\rightarrow$ sid

Flat

fid $\rightarrow$ fno

fid $\rightarrow$ size

fid $\rightarrow$ bid

fid $\rightarrow$ ownerid

Person

pid $\rightarrow$ name

pid $\rightarrow$ occupation

pid $\rightarrow$ category

pid $\rightarrow$ residenceid

PersonContact

Only trivial dependencies (no non-key attributes)

PersonEmail

Only trivial dependencies (no non-key attributes)

Branch

ifsc $\rightarrow$ bankname

ifsc $\rightarrow$ branchname

Account

accno $\rightarrow$ ifsc

accno $\rightarrow$ pid

Complaint

cid $\rightarrow$ description

cid $\rightarrow$ date

cid $\rightarrow$ status

cid $\rightarrow$ resolvedate

cid $\rightarrow$ pid

cid $\rightarrow$ sid

Visit

(pid, fid, entrydateandtime) $\rightarrow$ exitdateandtime

Maintenance

(fid, month) $\rightarrow$ amount

(fid, month) $\rightarrow$ status

(fid, month) $\rightarrow$ paymentdate

(fid, month) $\rightarrow$ paymentmethod

ParkingSlot

slotid $\rightarrow$ slotno

slotid $\rightarrow$ location

slotid $\rightarrow$ fid

Vehicle

vid → type

vid → model

vid → colour

vid → ownerid

vid → slotid

WorkOrder

(workerid, residentid, workdate) → type

(workerid, residentid, workdate) → cost

(workerid, residentid, workdate) → rating

Rental

(ownerid, tenantid, fid, startdate) → enddate

(ownerid, tenantid, fid, startdate) → depositamount

(ownerid, tenantid, fid, startdate) → rentamount

Amenity

aid → name

aid → location

aid → capacity

aid → sid

AmenityBooking

(aid, pid, bookingdate, startdatetime) → enddatetime

(aid, pid, bookingdate, startdatetime) → paymentmethod

(aid, pid, bookingdate, startdatetime) → status

(aid, pid, bookingdate, startdatetime) → fee

Manages

(sid, pid, tenurestartdate) → tenureenddate

(sid, pid, tenurestartdate) → role

Guards

(pid, bid, startdate) → enddate

(pid, bid, startdate) → shift

(pid, bid, startdate) → salary

3. BCNF Proof:

A relation is in BCNF if for every non-trivial FD $X \rightarrow Y$, X is a superkey.

In all the relations, the left-hand side of every non-trivial functional dependency is the primary key.

Since the primary key is a superkey, all relations satisfy the BCNF condition.

Hence, **all relations are in BCNF.**

From the ER diagram, the following relation was initially obtained:
Person(pid, name, occupation, category, residenceid, accno, ifsc, bankname, branchname)

FD Set:

pid $\rightarrow$ name
pid $\rightarrow$ occupation
pid $\rightarrow$ category
pid $\rightarrow$ residenceid
accno $\rightarrow$ ifsc
accno $\rightarrow$ pid
ifsc $\rightarrow$ bankname
ifsc $\rightarrow$ branchname

BCNF Condition Violating FDs:

pid $\rightarrow$ name
pid $\rightarrow$ occupation
pid $\rightarrow$ category
pid $\rightarrow$ residenceid

pid is not a superkey.

ifsc $\rightarrow$ bankname
ifsc $\rightarrow$ branchname

Ifsc is not a superkey.

so Person is not in BCNF.

Decomposition to BCNF:

Step 1: Decompose on ifsc $\rightarrow$ bankname, branchname

Create: Branch(ifsc, bankname, branchname)

Remaining Relation: R1(pid, name, occupation, category, residenceid, accno, ifsc)

Step 2: Decompose on pid $\rightarrow$ name, occupation, category, residenceid

Create: Person(pid, name, occupation, category, residenceid)

Remaining Relation: R2(pid, accno, ifsc)

Step 3: Final Relation

Account(accno, pid, ifsc)

Final Decomposed Relations:

Branch(ifsc, bankname, branchname)

Person(pid, name, occupation, category, residenceid)

Account(accno, pid, ifsc)

These three are in BCNF as proved above.

4. DDL Script:

```
CREATE SCHEMA society_db;  
SET search_path TO society_db;
```

```
CREATE TABLE Society(  
    sid INT PRIMARY KEY,  
    name VARCHAR(100),  
    street VARCHAR(100),  
    city VARCHAR(50),  
    pin VARCHAR(10)  
);
```

```
CREATE TABLE Building(  
    bid INT PRIMARY KEY,  
    name VARCHAR(100),  
    nooffloors INT,  
    sid INT REFERENCES Society(sid)  
);
```

```
CREATE TABLE Notice(  
    nid INT PRIMARY KEY,  
    title VARCHAR(100),  
    date DATE,  
    description TEXT,  
    sid INT REFERENCES Society(sid)  
);
```

```
CREATE TABLE Flat(  
    fid INT PRIMARY KEY,  
    fno VARCHAR(10),  
    size VARCHAR(20),  
    bid INT REFERENCES Building(bid),  
    ownerid INT REFERENCES Person(pid) NOT NULL  
);
```



```
CREATE TABLE Person(  
    pid INT PRIMARY KEY,  
    name VARCHAR(100),  
    occupation VARCHAR(50),  
    category VARCHAR(50),  
    residenceid INT REFERENCES Flat(fid)  
);
```

```
CREATE TABLE Branch(  
    ifsc VARCHAR(20) PRIMARY KEY,  
    bankname VARCHAR(100),  
    branchname VARCHAR(100)  
);
```

```
CREATE TABLE Account(  
    accno VARCHAR(30) PRIMARY KEY,  
    ifsc VARCHAR(20) REFERENCES Branch(ifsc),  
    pid INT REFERENCES Person(pid)  
);
```

```
CREATE TABLE PersonContact(  
    pid INT REFERENCES Person(pid),  
    contactno VARCHAR(15),  
    PRIMARY KEY(pid, contactno)  
);
```

```
CREATE TABLE PersonEmail(  
    pid INT REFERENCES Person(pid),  
    email VARCHAR(100),  
    PRIMARY KEY(pid, email)  
);
```

```
CREATE TABLE Complaint(  
    cid INT PRIMARY KEY,
```

```
description TEXT,  
date DATE,  
status VARCHAR(20),  
resolvedate DATE,  
pid INT REFERENCES Person(pid),  
sid INT REFERENCES Society(sid)  
);
```

```
CREATE TABLE Visit(  
pid INT REFERENCES Person(pid),  
fid INT REFERENCES Flat(fid),  
entrydatetime TIMESTAMP,  
exitdatetime TIMESTAMP,  
PRIMARY KEY(pid, fid, entrydatetime)  
);
```

```
CREATE TABLE Maintenance(  
fid INT REFERENCES Flat(fid),  
month VARCHAR(10),  
amount NUMERIC(10,2),  
status VARCHAR(20),  
paymentdate DATE,  
paymentmethod VARCHAR(30),  
PRIMARY KEY(fid, month)  
);
```

```
CREATE TABLE ParkingSlot(  
slotid INT PRIMARY KEY,  
slotno VARCHAR(10),  
location VARCHAR(50),  
fid INT REFERENCES Flat(fid)  
);
```

```
CREATE TABLE Vehicle(  
vid INT PRIMARY KEY,
```

```
type VARCHAR(30),  
model VARCHAR(50),  
colour VARCHAR(30),  
ownerid INT REFERENCES Person(pid) NOT NULL,  
slotid INT REFERENCES ParkingSlot(slotid)  
);
```

```
CREATE TABLE WorkOrder(  
    workerid INT REFERENCES Person(pid),  
    residentid INT REFERENCES Person(pid),  
    workdate DATE,  
    type VARCHAR(50),  
    cost NUMERIC(10,2),  
    rating INT,  
    PRIMARY KEY(workerid, residentid, workdate)  
);
```

```
CREATE TABLE Rental(  
    ownerid INT REFERENCES Person(pid),  
    tenantid INT REFERENCES Person(pid),  
    fid INT REFERENCES Flat(fid),  
    startdate DATE,  
    enddate DATE,  
    depositamount NUMERIC(10,2),  
    rentamount NUMERIC(10,2),  
    PRIMARY KEY(ownerid, tenantid, fid, startdate)  
);
```

```
CREATE TABLE Amenity(  
    aid INT PRIMARY KEY,  
    name VARCHAR(100),  
    location VARCHAR(50),  
    capacity INT,  
    sid INT REFERENCES Society(sid) NOT NULL  
);
```

```
CREATE TABLE AmenityBooking(  
    aid INT REFERENCES Amenity(aid),  
    pid INT REFERENCES Person(pid),  
    bookingdate DATE,  
    startdatetime TIMESTAMP,  
    enddatetime TIMESTAMP,  
    paymentmethod VARCHAR(30),  
    paymentstatus VARCHAR(20),  
    fee NUMERIC(10,2),  
    PRIMARY KEY(aid, pid, bookingdate, startdatetime)  
);
```

```
CREATE TABLE Manages(  
    sid INT REFERENCES Society(sid),  
    pid INT REFERENCES Person(pid),  
    tenurestartdate DATE,  
    tenureenddate DATE,  
    role VARCHAR(50),  
    PRIMARY KEY(sid, pid, tenurestartdate)  
);
```

```
CREATE TABLE Guards(  
    pid INT REFERENCES Person(pid),  
    bid INT REFERENCES Building(bid),  
    startdate DATE,  
    enddate DATE,  
    shift VARCHAR(20),  
    salary NUMERIC(10,2),  
    PRIMARY KEY(pid, bid, startdate)  
);
```